Chapter ML:III

III. Decision Trees

- Decision Trees Basics
- Impurity Functions
- Decision Tree Algorithms
- Decision Tree Pruning

©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting

Definition 10 (Overfitting)

Let D be a set of examples and let H be a hypothesis space. The hypothesis $h \in H$ is considered to overfit D if an $h' \in H$ with the following property exists:

$$\mathit{Err}(h, D) < \mathit{Err}(h', D) \quad \text{and} \quad \mathit{Err}^*(h) > \mathit{Err}^*(h'),$$

where $\mathit{Err}^*(h)$ denotes the true misclassification rate of h, while $\mathit{Err}(h, D)$ denotes the error of h on the example set D.

©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting

Definition 10 (Overfitting)

Let D be a set of examples and let H be a hypothesis space. The hypothesis $h \in H$ is considered to overfit D if an $h' \in H$ with the following property exists:

$$\mathit{Err}(h, D) < \mathit{Err}(h', D) \quad \text{and} \quad \mathit{Err}^*(h) > \mathit{Err}^*(h'),$$

where $\mathit{Err}^*(h)$ denotes the true misclassification rate of h, while $\mathit{Err}(h, D)$ denotes the error of h on the example set D.

Reasons for overfitting are often rooted in the example set D:

- D is noisy and we "learn noise"
- D is biased and hence not representative
- D is too small and hence pretends unrealistic data properties

©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting (continued)

Let $D_{tr} \subset D$ be the training set. Then $Err^*(h)$ can be estimated with a test set $D_{ts} \subset D$ where $D_{ts} \cap D_{tr} = \emptyset$ [holdout estimation]. The hypothesis $h \in H$ is considered to overfit D if an $h' \in H$ with the following property exists:

$$Err(h, D_{tr}) < Err(h', D_{tr}) \quad \text{and} \quad Err(h, D_{ts}) > Err(h', D_{ts})$$

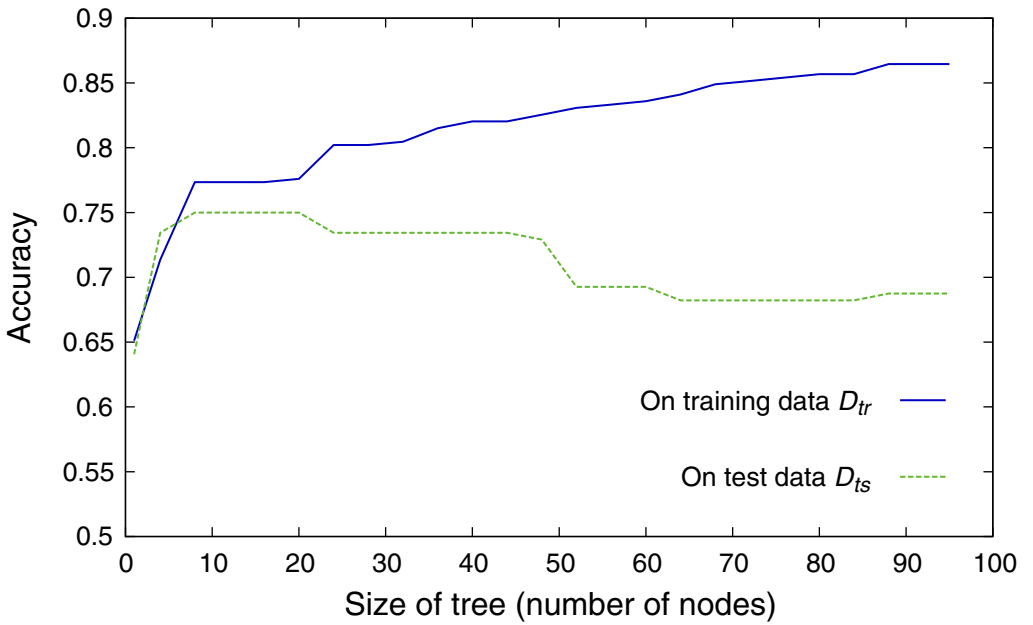
©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting (continued)

Let $D_{tr} \subset D$ be the training set. Then $Err^*(h)$ can be estimated with a test set $D_{ts} \subset D$ where $D_{ts} \cap D_{tr} = \emptyset$ [holdout estimation]. The hypothesis $h \in H$ is considered to overfit D if an $h' \in H$ with the following property exists:

$$Err(h, D_{tr}) < Err(h', D_{tr}) \quad \text{and} \quad Err(h, D_{ts}) > Err(h', D_{ts})$$



[Mitchell 1997]

© STEIN/LETTMANN 2021

Remarks:

- Accuracy is the percentage of correctly classified examples.
- When does $\mathit{Err}(T, D_{tr})$ of a decision tree T become zero?
- The training error $\mathit{Err}(T, D_{tr})$ of a decision tree T is a monotonically decreasing function in the size of T. See the following Lemma.

©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting (continued)

Lemma 11

Let t be a node in a decision tree T. Then, for each induced splitting $D(t_1), \ldots, D(t_s)$ of a set of examples $D(t)$ holds:

$$\mathit{Err}_{\mathit{cost}}(t, D(t)) \geq \sum_{i \in \{1,\ldots,s\}} \mathit{Err}_{\mathit{cost}}(t_i, D(t_i))$$

The equality is given in the case that all nodes $t, t_1, \ldots, t_s$ represent the same class.

©STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting (continued)

Proof (sketch)

$$
\begin{aligned}
\mathit{Err}_{\mathit{cost}}(t, D(t)) &= \min_{c' \in C} \sum_{c \in C} p(c \mid t) \cdot p(t) \cdot \mathit{cost}(c' \mid c) \\
&= \sum_{c \in C} p(c, t) \cdot \mathit{cost}(\mathit{label}(t) \mid c) \\
&= \sum_{c \in C} (p(c, t_1) + \ldots + p(c, t_{k_s})) \cdot \mathit{cost}(\mathit{label}(t) \mid c) \\
&= \sum_{i \in \{1,\ldots,k_s\}} \sum_{c \in C} (p(c, t_i) \cdot \mathit{cost}(\mathit{label}(t) \mid c)
\end{aligned}
$$

$$
\mathit{Err}_{\mathit{cost}}(t, D(t)) - \textstyle\sum_{i \in \{1,\ldots,k_s\}} \mathit{Err}_{\mathit{cost}}(t_i, D(t_i)) =
$$

$$
\sum_{i \in \{1,\ldots,k_s\}} \left(\sum_{c \in C} p(c, t_i) \cdot \mathit{cost}(\mathit{label}(t) \mid c) - \min_{c' \in C} \sum_{c \in C} p(c, t_i) \cdot \mathit{cost}(c' \mid c) \right)
$$

Observe that the summands on the right equation side are greater than or equal to zero.

©STEIN/LETTMANN 2021

Remarks:

- The lemma does also hold if the misclassification rate is used to evaluate effectiveness.
- The algorithm template for the construction of decision trees, *DT-construct*, prefers larger trees, entailing a more fine-grained splitting of D. A consequence of this behavior is a tendency to overfitting.

© STEIN/LETTMANN 2021

Decision Tree Pruning

Overfitting (continued)

Approaches to counter overfitting:

(a) Stopping of the decision tree construction process during training.

(b) Pruning of a decision tree after training:

- Splitting of D into three sets for training, validation, and test:
  - reduced error pruning
  - minimal cost complexity pruning
  - rule post pruning
- statistical tests such as χ^2 to assess generalization capability
- heuristic pruning

©STEIN/LETTMANN 2021

Decision Tree Pruning

(a) Stopping

Possible criteria for stopping [splitting criteria]:

1. Size of $D(t)$.
   $D(t)$ is not split if $|D(t)|$ is below a threshold.

2. Purity of $D(t)$.
   $D(t)$ is not split if all examples in $D(t)$ are members of the same class.

3. Impurity reduction of $D(t)$.
   $D(t)$ is not split if the resulting impurity reduction, $\Delta\iota$, is below a threshold.

Problems:

ad 1) A threshold that is too small results in oversized decision trees.

ad 1) A threshold that is too large omits useful splittings.

ad 2) Perfect purity cannot be expected with noisy data.

ad 3) $\Delta\iota$ cannot be extrapolated with regard to the tree height.

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning

The pruning principle:

1. Construct a sufficiently large decision tree T_{max}.
2. Prune T_{max}, starting from the leaf nodes upwards to the tree root.

Each leaf node t of T_{max} fulfills one or more of the following conditions:

- $D(t)$ is sufficiently small. Typically, $|D(t)| \leq 5$.
- $D(t)$ is pure.
- $D(t)$ is comprised of examples with identical feature vectors.

©STEIN/LETTMANN 2021

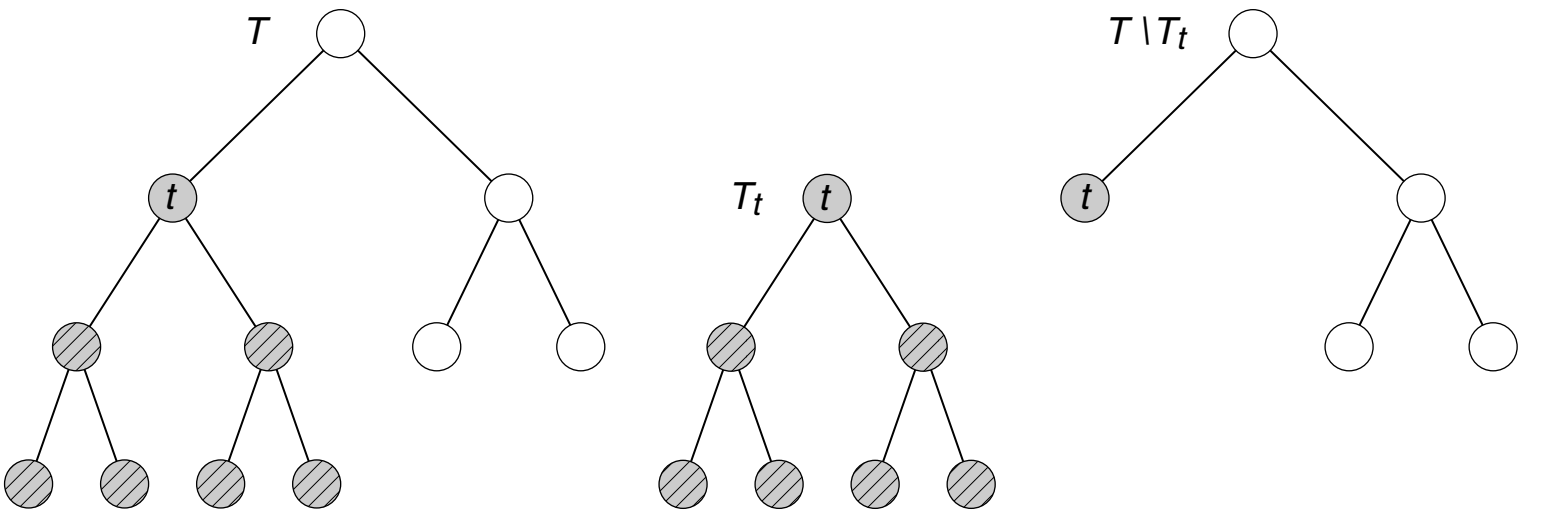
Decision Tree Pruning

(b) Pruning (continued)

Definition 12 (Decision Tree Pruning)

Given a decision tree T and an inner (non-root, non-leaf) node t. Then pruning of T with regard to t is the deletion of all successor nodes of t in T. The pruned tree is denoted as $T \setminus T_t$. The node t becomes a leaf node in $T \setminus T_t$.

Illustration:



© STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning (continued)

Definition 13 (Pruning-Induced Ordering)

Let T' and T be two decision trees. Then $T' \preceq T$ denotes the fact that T' is the result of a (possibly repeated) pruning applied to T. The relation $\preceq$ forms a partial ordering on the set of all trees.

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning (continued)

Definition 13 (Pruning-Induced Ordering)

Let T' and T be two decision trees. Then $T' \preceq T$ denotes the fact that T' is the result of a (possibly repeated) pruning applied to T. The relation $\preceq$ forms a partial ordering on the set of all trees.

Problems when assessing pruning candidates:

- Pruned decision trees may not stand in the $\preceq$-relation.
- Locally optimum pruning decisions may not result in the best candidates.
- Its monotonicity disqualifies $\mathit{Err}(T, D_{tr})$ as an estimator for $\mathit{Err}^*(T)$. [Lemma]

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning (continued)

Definition 13 (Pruning-Induced Ordering)

Let T' and T be two decision trees. Then $T' \preceq T$ denotes the fact that T' is the result of a (possibly repeated) pruning applied to T. The relation $\preceq$ forms a partial ordering on the set of all trees.

Problems when assessing pruning candidates:

- Pruned decision trees may not stand in the $\preceq$-relation.
- Locally optimum pruning decisions may not result in the best candidates.
- Its monotonicity disqualifies $\mathit{Err}(T, D_{tr})$ as an estimator for $\mathit{Err}^*(T)$. [Lemma]

Control pruning with validation set D_{vd}, where $D_{vd} \cap D_{tr} = \emptyset$, $D_{vd} \cap D_{ts} = \emptyset$:

1. $D_{tr} \subset D$ for decision tree construction.
2. $D_{vd} \subset D$ for overfitting analysis *during* pruning.
3. $D_{ts} \subset D$ for decision tree evaluation *after* pruning.

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning: Reduced Error Pruning

Steps of reduced error pruning :

1. $T = T_{\text{max}}$
2. Choose an inner node t in T.
3. Perform a tentative pruning of T with regard to t : $T' = T \setminus T_t$.
   Based on $D(t)$ assign class to t. [*DT-construct*]
4. If *Err*$(T', D_{vd}) \leq$ *Err*(T, D_{vd}) then accept pruning: $T = T'$.
5. Continue with Step 2 until all inner nodes of T are tested.

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning: Reduced Error Pruning

Steps of reduced error pruning :

1. $T = T_{\text{max}}$
2. Choose an inner node t in T.
3. Perform a tentative pruning of T with regard to t : $T' = T \setminus T_t$.
   Based on $D(t)$ assign class to t. [*DT-construct*]
4. If *Err*$(T', D_{vd}) \leq$ *Err*(T, D_{vd}) then accept pruning: $T = T'$.
5. Continue with Step 2 until all inner nodes of T are tested.

Problem:

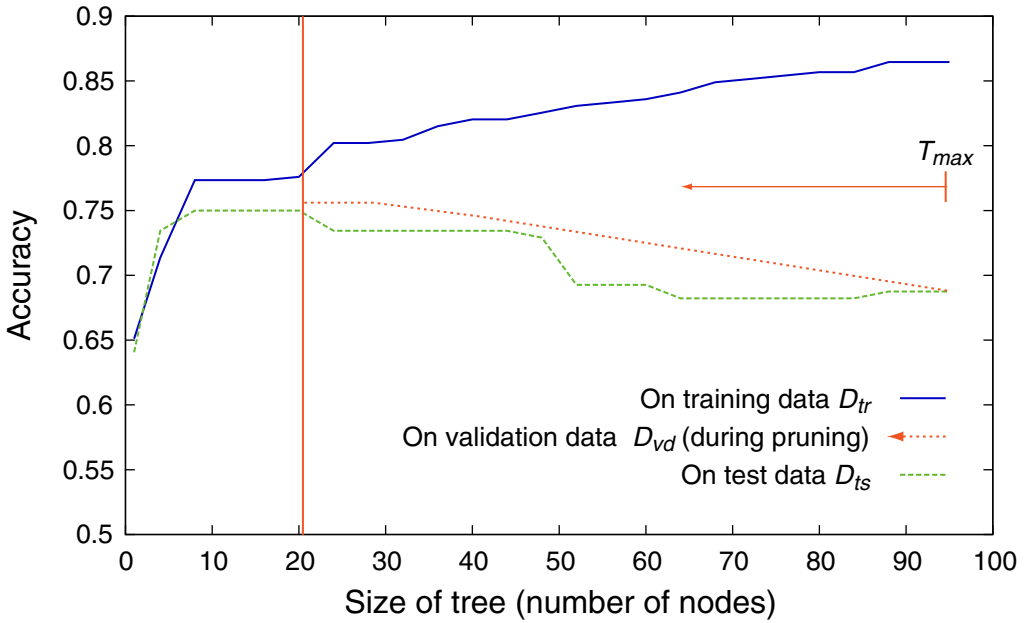
If D is small, its partitioning into three sets for training, validation, and test will discard valuable information for decision tree construction.

Improvement: rule post pruning

©STEIN/LETTMANN 2021

Decision Tree Pruning

(b) Pruning: Reduced Error Pruning (continued)



[Mitchell 1997]

© STEIN/LETTMANN 2021

Decision Tree Pruning

Extensions

- consideration of the misclassification cost introduced by a splitting
- “surrogate splittings” for insufficiently covered feature domains
- splittings based on (linear) combinations of features
- regression trees

© STEIN/LETTMANN 2021